

Hash me if you can: let's talk
about Python dictionaries!

Mia Bajić

What if he did this?

```
1 >>> class Key:
2 ...     def __init__(self, val):
3 ...         self.val = val
4 ...
5 ...     def __hash__(self):
6 ...         return 42
7 ...
8 ...     def __eq__(self, other):
9 ...         return self.val == other.val
10 ...
```

Let's build a dict of 10 000 of them

```
1 >>> import time
2 ...
3 ... keys = [Key(i) for i in range(10_000)]
4 ...
5 ... start = time.perf_counter()
6 ... d = {k: i for i, k in enumerate(keys)}
7 ... elapsed = time.perf_counter() - start
8 ...
9 ... print(f"{elapsed * 1000:.1f} ms")
```

Let's build a dict of 10 000 of them

```
1 >>> import time
2 ...
3 ... keys = [Key(i) for i in range(10_000)]
4 ...
5 ... start = time.perf_counter()
6 ... d = {k: i for i, k in enumerate(keys)}
7 ... elapsed = time.perf_counter() - start
8 ...
9 ... print(f"{elapsed * 1000:.1f} ms")
10 ...
11
12 3302.6 ms
```

Let's build a dict of 10 000 of them

```
1 >>> class Key:
2 ...     def __init__(self, val):
3 ...         self.val = val
4 ...
5 ...     def __hash__(self):
6 ...         return hash(self.val)
7 ...
8 ...     def __eq__(self, other):
9 ...         return self.val == other.val
10 ...
11 >>>
```

Let's build a dict of 10 000 of them

```
1 >>> import time
2 ...
3 ... keys = [Key(i) for i in range(10_000)]
4 ...
5 ... start = time.perf_counter()
6 ... d = {k: i for i, k in enumerate(keys)}
7 ... elapsed = time.perf_counter() - start
8 ...
9 ... print(f"{elapsed * 1000:.1f} ms")
```

Let's build a dict of 10 000 of them

```
1 >>> import time
2 ...
3 ... keys = [Key(i) for i in range(10_000)]
4 ...
5 ... start = time.perf_counter()
6 ... d = {k: i for i, k in enumerate(keys)}
7 ... elapsed = time.perf_counter() - start
8 ...
9 ... print(f"{elapsed * 1000:.1f} ms")
10 ...
11 2.9 ms
```

Let's start with something simpler

```
1 >>> my_list = ["a", "b", "c", "d", "e", "f"]
2 >>> my_list[4]
3 'e'
```

Python's answer: hash()

```
1 >>> hash("hi!")
2 -5223265829204838547
3 >>> hash(42)
4 42
5 >>> hash((1, 2, 3))
6 529344067295497451
```

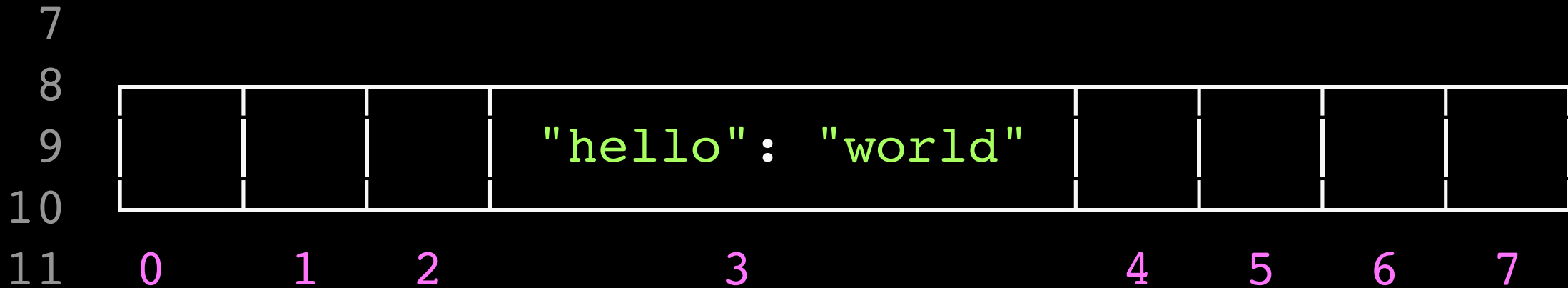
But the table isn't infinite

```
1 >>> hash("hi!")
2 -5223265829204838547
3 >>> table_size = 8
4 >>> -5223265829204838547 % 8
5 5
```

Note: CPython uses $\text{hash} \& (\text{size} - 1)$

Inserting a key

```
1 >>> d = {}  
2 >>> d["hello"] = "world"  
3  
4 1. hash("hello") = 1810699669017143179  
5 2. 1810...3179 % 8 = 3  
6 3. store ("hello", "world") in slot 3
```



Looking up a key

1 `d["hello"] → ?`

2

3 1. `hash("hello") = 1810699669017143179`

4 2. `1810...3179 % 8 = 3`

5 3. `read slot 3 → ("hello", "world")`

6 4. `return "world"`

7

8

9

			"hello": "world"				
--	--	--	------------------	--	--	--	--

10

11 0

1

2

3

4

5

6

7

12



The real picture (since 3.6)

```
1 >>> d = {}
2 >>> d["hello"] = "world"
3 >>> d["foo"] = "bar"
4
5 indices (hash-indexed, mostly empty):
```

6							
7	--	1	--	0	--	--	--
8							
9	0	1	2	3	4	5	6

```
10
11 entries (dense, insertion order):
```

12	
13	0: hash, "hello", "world"
14	1: hash, "foo", "bar"
15	

One small problem

1 hash("hello") % 8 = 3

2 hash("password") % 8 = 3

3

4

5

6

7



0

1

2

3

4

5

6

7

How many people in a room
before two of them share a
birthday?

Collisions happen way sooner than you'd think

```
1  23 people → 50% chance  
2  50 people → 97% chance  
3  70 people → 99.9% chance
```

A dict has the same problem

```
1 Table with 1 000 000 slots:
```

```
2
```

```
3 1,000 items → 40% chance of a collision
```

```
4 1,200 items → 50% chance of a collision
```

```
5 5,000 items → essentially guaranteed
```

Two strategies

1 Chaining

2 slot 3 → [A] → [B] → [C]

3

4

5 Open addressing

6 slot 3: A

7 slot 4: B

8 slot 5: C

Linear probing

- 1 Insert A: hash $\rightarrow$ slot 3. Empty. Go in slot 3.
- 2 Insert B: hash $\rightarrow$ slot 3. Taken. Try slot 4. Empty. Go in 4.
- 3 Insert C: hash $\rightarrow$ slot 3. Taken. Try slot 4. Taken. Try slot 5.
- 4 Insert D: hash $\rightarrow$ slot 4. Taken. Try slot 5. Taken. Try slot 6.

Python jumps around

```
perturb >>= PERTURB_SHIFT;  
i = mask & (i*5 + perturb + 1);
```

Back to our broken example

- 1 Same hash →
- 2 Same probe path →
- 3 Pile up linearly

Essentially every web framework had the same shape

```
1 @app.post("/signup")
2 def signup(form_data: dict):
3     ...
4
5 form_data = {}
6 for key, value in parse(request.body):
7     form_data[key] = value
```

Same broken dict, but the attacker is supplying the keys

```
1 key_0001 → hash mod 8 = 3
2 key_0002 → hash mod 8 = 3
3 key_0003 → hash mod 8 = 3
4 ...
5 key_9999 → hash mod 8 = 3
```

The old hash function was the problem

Two researchers walk into a German hacker conference...

Take home: Language Developers

Fix this – soon!

Randomize your hash functions!

28c3: Effective Denial of Service attacks against web application platforms



28c3
5.52K subscribers

Subscribe



264



Share



Save



Download



The numbers

The worst case in real life

200.000 multi-collisions à 10 bytes
roughly 2 MB

40.000.000.000 string comparisons
On a 1GHz machine, this is at least 40s

First fix: randomize the hash

Plot twist, one year later

Python 2.x and 3.x

- Randomization of hash() optional (-R)
- Instantaneous key recovery
- Multicollisions with TMTO

29C3: Hash-flooding DoS reloaded: attacks and defenses (EN)



Christiaan008
73.8K subscribers

Subscribe



39



Share



Save



Download



The actual fix: SipHash

SipHash **initialization**

256-bit state v_0 v_1 v_2 v_3

128-bit key k_0 k_1

$$v_0 = k_0 \oplus 0x736f6d6570736575$$

$$v_1 = k_1 \oplus 0x646f72616e646f6d$$

$$v_2 = k_0 \oplus 0x6c7967656e657261$$

$$v_3 = k_1 \oplus 0x7465646279746573$$

29C3: Hash-flooding DoS reloaded: attacks and defenses (EN)

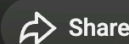


Christiaan008
73.8K subscribers

Subscribe



39



Share



Save



Download



You can see the randomization right now

```
1 mia@mias-MacBook-Pro ~ % python3 -c 'print(hash("hello"))'
2 585605004346668881
3 mia@mias-MacBook-Pro ~ % python3 -c 'print(hash("hello"))'
4 3473006783256160617
5 mia@mias-MacBook-Pro ~ % python3 -c 'print(hash("hello"))'
6 -732890651206170119
7 mia@mias-MacBook-Pro ~ % python3 -c 'print(hash("hello"))'
8 -561757827254387862
```

Weirder than it has to be

```
1 >>> hash(-1)
2 -2
3 >>> hash(-2)
4 -2
```

-1 is reserved for "something broke"

```
if (_PyLong_IsCompact(v)) {  
    x = (Py_uhash_t)_PyLong_CompactValue(v);  
    if (x == (Py_uhash_t)-1) {  
        x = (Py_uhash_t)-2;  
    }  
    return x;  
}
```

What happens when you delete a key?

```
1 >>> d = {"a": 1, "b": 2, "c": 3}
2 >>> del d["b"]
```

An empty slot would break probing

```
1 slot 3: KEY_A
2 slot 4: KEY_B
3 slot 5: KEY_C
4
5 slot 3: KEY_A
6 slot 4: empty
7 slot 5: KEY_C # unreachable
```

Mark the slot as "used to be something"

```
1 slot 3: KEY_A  
2 slot 4: tombstone  
3 slot 5: KEY_C
```

Hash and equality have to agree

```
1  if a == b then hash(a) == hash(b)
```

Why this matters

```
1 >>> class Point:
2 ...     def __init__(self, x, y):
3 ...         self.x, self.y = x, y
4 ...     def __eq__(self, other):
5 ...         return (self.x, self.y) == (other.x,
6 ...         other.y)
```

Why this matters

```
1 >>> p1 = Point(1, 2)
2 ... p2 = Point(1, 2)
3 ...
4 >>> d = {p1: "first"}
5 Traceback (most recent call last):
6   File "<python-input-20>", line 1, in <module>
7     d = {p1: "first"}
8         ^^^^^^^^^^^
9 TypeError: cannot use 'Point' as a dict key
   (unhashable type: 'Point')
```

Beyond dicts

Hashing is everywhere once you look

1	Bloom filters	→	"is X probably in this set?"
2			Chrome safe browsing, Cassandra
3			
4	Consistent hashing	→	routing without re-melting caches
5			every CDN, DynamoDB, Memcached
6			
7	Merkle trees	→	tamper-evident data
8			git, blockchains, IPFS
9			
10	HyperLogLog	→	count unique things in tiny memory
11			Redis PFCOUNT, BigQuery

Wrapping up

hash $\rightarrow$ mod $\rightarrow$ slot

Wrapping up

hash $\rightarrow$ mod $\rightarrow$ slot

The dict is only as good as its hash

Wrapping up

hash $\rightarrow$ mod $\rightarrow$ slot

The dict is only as good as its hash

Hashing is bigger than dicts only

Thank you for your attention!



slides



LinkedIn



Instagram



my podcast